



OKX - Smart Wallet

Security Assessment

CertiK Assessed on Sept 26th, 2025





Certik Assessed on Sept 26th, 2025

OKX - Smart Wallet

The security assessment was prepared by Certik.

Executive Summary

TYPES

Wallet

ECOSYSTEM

EVM Compatible

METHODS

Manual Review, Static Analysis

LANGUAGE

Solidity

TIMELINE

Preliminary comments published on 09/26/2025

Final report published on 09/26/2025

Vulnerability Summary



10

Total Findings

9

Resolved

0

Partially Resolved

1

Acknowledged

0

Declined



0 Centralization

Centralization findings highlight privileged roles & functions and their capabilities, or instances where the project takes custody of users' assets.



0 Critical

Critical risks are those that impact the safe functioning of a platform and must be addressed before launch. Users should not invest in any project with outstanding critical risks.



0 Major

Major risks may include logical errors that, under specific circumstances, could result in fund losses or loss of project control.



1 Medium

1 Resolved



Medium risks may not pose a direct risk to users' funds, but they can affect the overall functioning of a platform.



2 Minor

2 Resolved



Minor risks can be any of the above, but on a smaller scale. They generally do not compromise the overall integrity of the project, but they may be less efficient than other solutions.



7 Informational

6 Resolved, 1 Acknowledged



Informational errors are often recommendations to improve the style of the code or certain operations to fall within industry best practices. They usually do not affect the overall functioning of the code.

TABLE OF CONTENTS | OKX - SMART WALLET

Summary

[Executive Summary](#)

[Vulnerability Summary](#)

[Codebase](#)

[Audit Scope](#)

[Approach & Methods](#)

Findings

[OEA-05 : Potential Issues With Unprotected Initializer For EIP-7702](#)

[OEA-06 : Incorrect Event Semantics In `createAccount` Function Can Mislead Off-Chain Consumers](#)

[OEA-07 : Inconsistent Validator Check In `removeOwner\(\)` Function](#)

[OEA-08 : Missing Zero Address Validation](#)

[OEA-09 : Missing `settings` Field In `OwnerAdded` And `OwnerUpdated` Events Reduces Transparency](#)

[OEA-10 : Third-Party Dependencies](#)

[OEA-11 : Entrypoint Contract Compatibility Issue](#)

[OEA-12 : Inaccurate Error Message And Code Comments](#)

[OEA-13 : Unnecessary Unchecked Increment In For Loop](#)

[OEA-14 : Custom Errors Can Be Used With Require Statements Since Solidity Version 0.8.27](#)

Optimizations

[OEA-01 : User-Defined Getters](#)

[OEA-02 : `getVerifiedValidator\(\)` Does Not Conform To Its Specifications](#)

[OEA-03 : Inefficient Use of `decodeCalls\(\)` Function](#)

[OEA-04 : AA-Wallet: Existence of Unused Code](#)

Appendix

Disclaimer

CODEBASE | OKX - SMART WALLET

Repository

<https://github.com/okx/okx-smart-wallet>

Commit

Base evm-AA-wallet: [f104cdc2b3a1999f1d5c97b26f14dfe599f148ec](#)

Update1 evm-AA-wallet: [94a25211094e1820e00f34483545159a4d02dfb7](#)

Final evm-AA-wallet: [c935ece44678bf06a294ccf1edaed4507e41452e](#)

Audit Scope

The file in scope is listed in the appendix.

APPROACH & METHODS | OKX - SMART WALLET

This audit was conducted for OKX to evaluate the security and correctness of the smart contracts associated with the OKX - Smart Wallet project. The assessment included a comprehensive review of the in-scope smart contracts. The audit was performed using a combination of Manual Review and Static Analysis.

The review process emphasized the following areas:

- Architecture review and threat modeling to understand systemic risks and identify design-level flaws.
- Identification of vulnerabilities through both common and edge-case attack vectors.
- Manual verification of contract logic to ensure alignment with intended design and business requirements.
- Dynamic testing to validate runtime behavior and assess execution risks.
- Assessment of code quality and maintainability, including adherence to current best practices and industry standards.

The audit resulted in findings categorized across multiple severity levels, from informational to critical. To enhance the project's security and long-term robustness, we recommend addressing the identified issues and considering the following general improvements:

- Improve code readability and maintainability by adopting a clean architectural pattern and modular design.
- Strengthen testing coverage, including unit and integration tests for key functionalities and edge cases.
- Maintain meaningful inline comments and documentations.
- Implement clear and transparent documentation for privileged roles and sensitive protocol operations.
- Regularly review and simulate contract behavior against newly emerging attack vectors.

FINDINGS | OKX - SMART WALLET



10
Total Findings

0
Critical

0
Centralization

0
Major

1
Medium

2
Minor

7
Informational

This report has been prepared for OKX to identify potential vulnerabilities and security issues within the reviewed codebase. During the course of the audit, a total of 10 issues were identified. Leveraging a combination of Manual Review & Static Analysis the following findings were uncovered:

ID	Title	Category	Severity	Status
OEA-05	Potential Issues With Unprotected Initializer For EIP-7702	Logical Issue	Medium	● Resolved
OEA-06	Incorrect Event Semantics In <code>createAccount</code> Function Can Mislead Off-Chain Consumers	Design Issue	Minor	● Resolved
OEA-07	Inconsistent Validator Check In <code>removeOwner()</code> Function	Inconsistency	Minor	● Resolved
OEA-08	Missing Zero Address Validation	Volatile Code	Informational	● Resolved
OEA-09	Missing <code>settings</code> Field In <code>OwnerAdded</code> And <code>OwnerUpdated</code> Events Reduces Transparency	Logical Issue	Informational	● Resolved
OEA-10	Third-Party Dependencies	Volatile Code	Informational	● Acknowledged
OEA-11	Entrypoint Contract Compatibility Issue	Design Issue, Inconsistency	Informational	● Resolved
OEA-12	Inaccurate Error Message And Code Comments	Coding Issue	Informational	● Resolved
OEA-13	Unnecessary Unchecked Increment In For Loop	Coding Style, Code Optimization	Informational	● Resolved
OEA-14	Custom Errors Can Be Used With Require Statements Since Solidity Version 0.8.27	Coding Style	Informational	● Resolved

OEA-05 | Potential Issues With Unprotected Initializer For EIP-7702

Category	Severity	Location	Status
Logical Issue	● Medium	SmartWallet.sol (evm-AA-wallet): 78	● Resolved

Description

The intended flow for `EIP-7702` account is for them to set their code with the `SmartWallet` and to immediately initialize it (for example this is done in the script `SetCodeAndInitialize`). However, in practice users may not always call `initialize()`, leaving it able to be called by any entity that can interact with the EOA.

Recommendation

We recommend adding access restriction to the `initialize()` function to ensure that if an EIP-7702 account fails to call it on themselves, that it cannot be called by another entity.

Alleviation

[Certik, 09/18/2025]: The client revised the code and resolved this issue in commit [5b0a90dcdaae5204d544c1007d1b3e2114f14e8a](#).

OEA-06 | Incorrect Event Semantics In `createAccount` Function Can Mislead Off-Chain Consumers

Category	Severity	Location	Status
Design Issue	● Minor	SmartWalletFactory.sol (evm-AA-wallet): 36	● Resolved

Description

The `createAccount` function is intended to create a deterministic smart wallet at a computed address. However, the function emits an `AccountCreated` event every time it is called, regardless of whether a new account is actually deployed or one already exists at that address. When the account already exists, the function makes no state changes but still emits the event, incorrectly signaling a new deployment.

This behavior introduces ambiguity for off-chain consumers—such as indexers, monitoring tools, or user interfaces—that rely on event logs to track actual wallet creation. These systems may interpret repeated `AccountCreated` events as multiple deployments, leading to inaccurate records, misleading user displays, or duplicate entries in indexing databases.

Recommendation

Modify the event emission logic to ensure `AccountCreated` is emitted only when a new wallet contract is deployed. If it is useful to track repeated calls, emit a separate event (e.g., `AccountCreationAttempt` or `AccountExists`) to distinguish these cases. This preserves clarity and ensures off-chain systems have reliable signals for wallet creation.

Alleviation

[CertiK, 09/18/2025]: The client revised the code and resolved this issue in commit [5c02344b64eb059d7873edb37477bc34b3bbb9dd](#).

OEA-07 | Inconsistent Validator Check In `removeOwner()` Function

Category	Severity	Location	Status
Inconsistency	Minor	OwnerManager.sol (evm-AA-wallet): 89	Resolved

Description

The `removeOwner(bytes32 keyHash)` function does not check if `_ownerKeys` contains the provided `keyHash` before proceeding to remove the key and emitting an `OwnerRemoved` event. This is inconsistent with the `_addOwner(bytes32 keyHash, address validator, uint256 settings)` function, which checks if `_ownerKeys` contains the `keyHash` before adding a new owner.

```
88 function removeOwner(bytes32 keyHash) external onlySelf {
89     emit OwnerRemoved(keyHash, ownerValidators[keyHash]);
90
91     _removeValidator(keyHash);
92 }
```

Having a check can prevent unnecessary or incorrect emissions of events while also avoiding erroneous removal of validators that do not exist.

Recommendation

It is recommended to add a check within the `removeOwner` function to verify if `_ownerKeys` contains the `keyHash` before emitting the event and removing the validator. This will ensure consistency with the `_addOwner` function and prevent unnecessary or incorrect operations.

Alleviation

[CertiK, 09/19/2025]: The client revised the code and resolved this issue in commit

[1e03e6526b8aef68b7ec7dffbaafaa89483fdd49](#).

OEA-08 | Missing Zero Address Validation

Category	Severity	Location	Status
Volatile Code	● Informational	AllowanceManager.sol (evm-AA-wallet): 32; SmartWalletFactory.sol (evm-AA-wallet): 15	● Resolved

Description

The cited address input is missing a check that it is not `address(0)`.

Recommendation

We recommend adding a check the passed-in address is not `address(0)` to prevent unexpected errors.

Alleviation

[OKX, 09/19/2025]: Fixed. Added validation to ensure tokenSpender address is not equal to address(0). For the implementation address in SmartWalletFactory, we will use deployment scripts to guarantee the implementation address is correct.

[CertiK, 09/18/2025]: The client revised the code or clarified they will use deployment scripts to ensure correctness and resolved this issue in commit [b377414b689ad97acaf961eede2a672be4497903](#).

OEA-09 Missing `settings` Field In `OwnerAdded` And `OwnerUpdated` Events Reduces Transparency

Category	Severity	Location	Status
Logical Issue	● Informational	OwnerManager.sol (evm-AA-wallet): 57, 82	● Resolved

Description

The contract emits `OwnerAdded` and `OwnerUpdated` events when ownership-related changes occur. These events currently log the `keyHash` and `validator`, but omit the `settings` field that also influences the owner configuration. As a result, off-chain consumers relying solely on events for monitoring or indexing cannot fully reconstruct the state of owner records.

This limitation weakens transparency and may complicate auditing, monitoring, or troubleshooting processes. Observers must read storage directly to obtain the full context, which is inefficient and less reliable.

Recommendation

Update both events to include the `settings` field, ensuring that event logs reflect all critical parameters. This improves observability, supports off-chain indexers, and provides a complete audit trail of owner configuration changes.

Alleviation

[CertiK, 09/18/2025]: The client revised the code and resolved this issue in commit [1373d6e563ad9912f82a07a53aa631e5447b3f5c](#).

OEA-10 | Third-Party Dependencies

Category	Severity	Location	Status
Volatile Code	● Informational	ERC4337Account.sol (evm-AA-wallet): 19; SmartWallet.sol (evm-AA-wallet): 194, 206; ValidationManager.sol (evm-AA-wallet): 57; libraries/PasskeyValidatorLib.sol (evm-AA-wallet): 66	● Acknowledged

Description

The project is serving as the underlying entity to interact with third-party contracts, **entryPoint**, **webAuthn**, **validator**, **hookAddress**. The scope of the audit treats third-party entities as black boxes and assumes their functional correctness. However, in the real world, third parties can be compromised and this may lead to lost or stolen assets.

Recommendation

We recommend that the project team constantly monitor the functionality of the dependency to mitigate any side effects that may occur when unexpected changes are introduced.

Alleviation

[OKX, 09/18/2025]: We use the official ERC-4337 EntryPoint address. The WebAuthn library has been audited. Validators and hook addresses are registered by users themselves, giving them full control over these components.

OEA-11 | Entrypoint Contract Compatibility Issue

Category	Severity	Location	Status
Design Issue, Inconsistency	● Informational	ERC4337Account.sol (evm-AA-wallet): 18~20; SmartWallet.sol (evm-AA-wallet): 101	● Resolved

Description

The `SmartWallet` contract indicates that "this contract is only compatible with Entrypoint versions v0.7.0 and v0.8.0". However, in the `ERC4337Account` contract which is inherited by `SmartWallet`, the `entryPoint()` is a fixed `0x0000000071727De22E5E9d8BAf0edAc6f37da032` address which corresponds to the `entrypoint` contract version 0.7.0. This means that the `SmartWallet` contract does not work with `Entrypoint` contract version 0.8.0, which is deployed at a different address (e.g. `0x4337084d9e255ff0702461cf8895ce9e3b5ff108` on ETH mainnet).

Recommendation

We'd like to understand if the project team intends to support both version 0.7.0 and 0.8.0, or only version 0.7.0 of the `Entrypoint` contract, and update either the code comment or the `entryPoint()` function accordingly.

Alleviation

[Certik, 09/18/2025]: We only support EntryPoint v0.7.0 as originally intended. The comment has been updated to reflect this in commit [a6876f9279c0d71d0ee1bd48f855f027065f2217](#).

OEA-12 | Inaccurate Error Message And Code Comments

Category	Severity	Location	Status
Coding Issue	● Informational	OwnerManager.sol (evm-AA-wallet): 85~92, 95; SmartWallet.sol (evm-AA-wallet): 68, 92	● Resolved

Description

There are several inaccurate error messages and code comments. For example, the `expiration` is a `timestamp` check that applies to the `validator` that corresponds to a owner's `keyHash`, and not the `owner` itself. The `execute()` function has a `onlyOwner` modifier, which not only allows self calls as the comment in line 92 indicates, but also allows valid `owner` as the caller. In `OwnerManager`, the `removeOwner()` function has a `onlySelf` modifier which only allows self call, as opposed to the wallet owner as the comment in line 86 indicates. The function also removes both the owner and its corresponding validator and settings.

Recommendation

We recommend that the team review the accuracy of code comments to ensure consistency with code implementation and the intended design.

Alleviation

[CertiK, 09/19/2025]: The client revised the code and resolved this issue in commit: [b287502e0864d461b5aaa5b984b9e6438d835510](https://github.com/okx/okx-smart-wallet/commit/b287502e0864d461b5aaa5b984b9e6438d835510).

OEA-13 | Unnecessary Unchecked Increment In For Loop

Category	Severity	Location	Status
Coding Style, Code Optimization	● Informational	AllowanceManager.sol (evm-AA-wallet): 42~44	● Resolved

Description

Throughout the codebase, it updates the increment `i` in an `unchecked` block within the `for` loop, which initially saved gas as this cannot overflow and can be placed in an unchecked block.

```
unchecked {  
    ++i;  
}
```

As of Solidity compiler version 0.8.22, it is now no longer necessary to explicitly make this optimization as it is handled automatically.

Recommendation

We recommend rewriting the `for` loop increment next to the lower and upper bound, to enhance readability, since this optimization is implicitly handled by the `for` loop structure in the chosen solidity version.

Alleviation

[CertiK, 09/19/2025]: The client revised the code and resolved this issue in commits

- [839dcf7f2a320ec5af672c7db106b422699833b8](#);
- [76b6d5978ca0d2b205ce5b65a1b8801d53afc3ac](#);
- [b287502e0864d461b5aaa5b984b9e6438d835510](#).

OEA-14 | Custom Errors Can Be Used With Require Statements Since Solidity Version 0.8.27

Category	Severity	Location	Status
Coding Style	● Informational	BaseAuthorization.sol (evm-AA-wallet): 10~12; ERC4337Account.sol (evm-AA-wallet): 13; OwnerManager.sol (evm-AA-wallet): 47~49, 71~73, 250~256; SmartWallet.sol (evm-AA-wallet): 62~64, 67~69, 132, 135~137, 143~150, 158, 161~168, 198~200	● Resolved

Description

Since solidity version 0.8.27 custom errors can be used with require statements in both the via IR and legacy pipeline. We point to those errors in contracts whose solidity version is greater than or equal to 0.8.27, however, the other contracts could also have their versions increased if require statements are desired to be consistent throughout the codebase.

Recommendation

We recommend using require statements for error handling to improve the readability of the codebase.

Alleviation

[OKX, 09/18/2025]: We prefer to use the `if-revert` pattern with custom errors instead of `require` statements. This provides better gas efficiency and clearer error handling throughout the codebase.

OPTIMIZATIONS | OKX - SMART WALLET

ID	Title	Category	Severity	Status
OEA-01	User-Defined Getters	Gas Optimization	Optimization	● Resolved
OEA-02	<code>getVerifiedValidator()</code> Does Not Conform To Its Specifications	Code Optimization	Optimization	● Resolved
OEA-03	Inefficient Use Of <code>decodeCalls()</code> Function	Code Optimization	Optimization	● Resolved
OEA-04	AA-Wallet: Existence Of Unused Code	Code Optimization	Optimization	● Acknowledged

OEA-01 | User-Defined Getters

Category	Severity	Location	Status
Gas Optimization	● Optimization	NonceManager.sol (evm-AA-wallet): 36~38	● Resolved

Description

The linked functions are equivalent to the compiler-generated getter functions for the respective variables.

Recommendation

We advise that the linked variables are instead declared as `public` as compiler-generated getter functions are less prone to error and much more maintainable than manually written ones.

Alleviation

[CertiK, 09/18/2025]: The client revised the code and resolved this issue in commit [c0cb8daa9154131c0da853adb39c2992b65c05a2](#).

OEA-02 `getVerifiedValidator()` Does Not Conform To Its Specifications

Category	Severity	Location	Status
Code Optimization	● Optimization	OwnerManager.sol (evm-AA-wallet): 168	● Resolved

Description

The `getVerifiedValidator()` function is intended to return the built-in ECDSA validator address (`address(1)`) for self-signing when no validator is installed, according to its specification and associated comments. However, the current implementation returns `address(0)` in this case. This discrepancy could cause incorrect validator selection logic elsewhere in the contract potentially impacting expected contract behavior.

Recommendation

We recommend that the code in `getVerifiedValidator()` be refactored to ensure it returns the built-in ECDSA validator address (`address(1)`) for self-signing, rather than returning `address(0)`. This will ensure proper compliance with the intended specification and prevent unexpected behavior.

Alleviation

[CertiK, 09/18/2025]: The client revised the code and resolved this issue in commit [4228b186e2b38713fc2c29ae7d6c0ff0c0f4919b](#).

OEA-03 | Inefficient Use Of `decodeCalls()` Function

Category	Severity	Location	Status
Code Optimization	● Optimization	libraries/DecodeLib.sol (evm-AA-wallet): 12~21	● Resolved

Description

The provided code uses a custom assembly block with the `memory-safe` marker to decode a `Call[]` data structure from the `callData`. However, this coding pattern directly relies on the specifics of the ABI encoding mechanism, which can introduce potential risks and inefficiencies.

```
9 function decodeCalls(  
10     bytes calldata callData  
11 ) internal pure returns (Call[] calldata calls) {  
12     assembly ("memory-safe") {  
13         let dataPointer := add(  
14             callData.offset,  
15             calldataload(callData.offset)  
16         )  
17  
18         // Extract Calls  
19         calls.offset := add(dataPointer, 32)  
20         calls.length := calldataload(dataPointer)  
21     }  
22 }
```

With Solidity's built-in function `abi.decode()`, decoding can be done much more safely and efficiently. Therefore, it is advisable to refactor your code and make use of the built-in decode function.

Recommendation

Refactor the `decodeCalls` function by replacing the assembly block with `abi.decode()`. This change will make your code safer and more efficient.

Here is the modified function:

```
9 function decodeCalls(  
10     bytes calldata callData  
11 ) internal pure returns (Call[] calldata calls) {  
12     calls = abi.decode(callData, (Call[]));  
13 }
```

Alleviation

[OKX, 09/18/2025]: We will not modify this implementation. The current assembly-based approach is intentional as it allows us to return `calldata` type directly, which is more gas-efficient. The suggested `abi.decode()` approach would only return `memory` type, requiring additional memory allocation and copying, thus increasing gas costs.

OEA-04 | AA-Wallet: Existence Of Unused Code

Category	Severity	Location	Status
Code Optimization	● Optimization	interfaces/IAllowanceManager.sol (evm-AA-wallet): 44	● Acknowledged

Description

The code provided contains elements that are not used.

- error IncorrectSender
- error TokenTransferFailed
- error BatchLengthMismatch

Recommendation

We recommend removing unused code.

Alleviation

[OKX, 09/18/2025]: Acknowledged. We agree these error definitions are redundant.

APPENDIX | OKX - SMART WALLET

Audit Scope

File

-  interfaces/IAllowanceManager.sol
-  libraries/PasskeyValidatorLib.sol
-  ERC4337Account.sol
-  SmartWallet.sol
-  ValidationManager.sol
-  libraries/DecodeLib.sol
-  AllowanceManager.sol
-  BaseAuthorization.sol
-  NonceManager.sol
-  OwnerManager.sol
-  SmartWalletFactory.sol
-  interfaces/IERC4337Account.sol
-  interfaces/IERC7201.sol
-  interfaces/IHook.sol
-  interfaces/INonceManager.sol
-  interfaces/IOwnerManager.sol
-  interfaces/ISmartWallet.sol
-  interfaces/ISmartWalletFactory.sol
-  interfaces/ISmartWalletSimulator.sol

File

 interfaces/IValidator.sol
 libraries/BatchedCallLib.sol
 libraries/CallLib.sol
 libraries/ChainlessLib.sol
 libraries/ECDSAValidatorLib.sol
 libraries/Errors.sol
 libraries/MerkleProofProcessor.sol
 libraries/MessageSignLib.sol
 libraries/Static.sol
 validator/ECDSAValidator.sol
 validator/PasskeyValidator.sol
 ERC712.sol
 ERC7201.sol
 ExecutionManager.sol
 FallbackHandler.sol
 OKXSmartWalletEntry.sol
 Types.sol

Finding Categories

Categories	Description
Gas Optimization	Gas Optimization findings do not affect the functionality of the code but generate different, more optimal EVM opcodes resulting in a reduction on the total gas cost of a transaction.

Categories	Description
Coding Style	Coding Style findings may not affect code behavior, but indicate areas where coding practices can be improved to make the code more understandable and maintainable.
Coding Issue	Coding Issue findings are about general code quality including, but not limited to, coding mistakes, compile errors, and performance issues.
Inconsistency	Inconsistency findings refer to different parts of code that are not consistent or code that does not behave according to its specification.
Volatile Code	Volatile Code findings refer to segments of code that behave unexpectedly on certain edge cases and may result in vulnerabilities.
Logical Issue	Logical Issue findings indicate general implementation issues related to the program logic.
Design Issue	Design Issue findings indicate general issues at the design level beyond program logic that are not covered by other finding categories.

DISCLAIMER | CERTIK

This report is subject to the terms and conditions (including without limitation, description of services, confidentiality, disclaimer and limitation of liability) set forth in the Services Agreement, or the scope of services, and terms and conditions provided to you ("Customer" or the "Company") in connection with the Agreement. This report provided in connection with the Services set forth in the Agreement shall be used by the Company only to the extent permitted under the terms and conditions set forth in the Agreement. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes, nor may copies be delivered to any other person other than the Company, without CertiK's prior written consent in each instance.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts CertiK to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. CertiK's position is that each company and individual are responsible for their own due diligence and continuous security. CertiK's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by CertiK is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

ALL SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF ARE PROVIDED "AS IS" AND "AS AVAILABLE" AND WITH ALL FAULTS AND DEFECTS WITHOUT WARRANTY OF ANY KIND. TO THE MAXIMUM EXTENT PERMITTED UNDER APPLICABLE LAW, CERTIK HEREBY DISCLAIMS ALL WARRANTIES, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE WITH RESPECT TO THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS. WITHOUT LIMITING THE FOREGOING, CERTIK SPECIFICALLY DISCLAIMS ALL IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITLE AND NON-INFRINGEMENT, AND ALL WARRANTIES ARISING FROM COURSE OF DEALING, USAGE, OR TRADE PRACTICE. WITHOUT LIMITING THE FOREGOING, CERTIK MAKES NO WARRANTY OF ANY KIND THAT THE SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF, WILL MEET CUSTOMER'S OR ANY OTHER PERSON'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULT, BE COMPATIBLE OR WORK WITH ANY SOFTWARE, SYSTEM, OR OTHER SERVICES, OR BE SECURE, ACCURATE, COMPLETE, FREE OF HARMFUL CODE, OR ERROR-FREE. WITHOUT LIMITATION TO THE FOREGOING, CERTIK PROVIDES NO WARRANTY OR

UNDERTAKING, AND MAKES NO REPRESENTATION OF ANY KIND THAT THE SERVICE WILL MEET CUSTOMER'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULTS, BE COMPATIBLE OR WORK WITH ANY OTHER SOFTWARE, APPLICATIONS, SYSTEMS OR SERVICES, OPERATE WITHOUT INTERRUPTION, MEET ANY PERFORMANCE OR RELIABILITY STANDARDS OR BE ERROR FREE OR THAT ANY ERRORS OR DEFECTS CAN OR WILL BE CORRECTED.

WITHOUT LIMITING THE FOREGOING, NEITHER CERTIK NOR ANY OF CERTIK'S AGENTS MAKES ANY REPRESENTATION OR WARRANTY OF ANY KIND, EXPRESS OR IMPLIED AS TO THE ACCURACY, RELIABILITY, OR CURRENCY OF ANY INFORMATION OR CONTENT PROVIDED THROUGH THE SERVICE. CERTIK WILL ASSUME NO LIABILITY OR RESPONSIBILITY FOR (I) ANY ERRORS, MISTAKES, OR INACCURACIES OF CONTENT AND MATERIALS OR FOR ANY LOSS OR DAMAGE OF ANY KIND INCURRED AS A RESULT OF THE USE OF ANY CONTENT, OR (II) ANY PERSONAL INJURY OR PROPERTY DAMAGE, OF ANY NATURE WHATSOEVER, RESULTING FROM CUSTOMER'S ACCESS TO OR USE OF THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS.

ALL THIRD-PARTY MATERIALS ARE PROVIDED "AS IS" AND ANY REPRESENTATION OR WARRANTY OF OR CONCERNING ANY THIRD-PARTY MATERIALS IS STRICTLY BETWEEN CUSTOMER AND THE THIRD-PARTY OWNER OR DISTRIBUTOR OF THE THIRD-PARTY MATERIALS.

THE SERVICES, ASSESSMENT REPORT, AND ANY OTHER MATERIALS HEREUNDER ARE SOLELY PROVIDED TO CUSTOMER AND MAY NOT BE RELIED ON BY ANY OTHER PERSON OR FOR ANY PURPOSE NOT SPECIFICALLY IDENTIFIED IN THIS AGREEMENT, NOR MAY COPIES BE DELIVERED TO, ANY OTHER PERSON WITHOUT CERTIK'S PRIOR WRITTEN CONSENT IN EACH INSTANCE.

NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS.

THE REPRESENTATIONS AND WARRANTIES OF CERTIK CONTAINED IN THIS AGREEMENT ARE SOLELY FOR THE BENEFIT OF CUSTOMER. ACCORDINGLY, NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH REPRESENTATIONS AND WARRANTIES AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH REPRESENTATIONS OR WARRANTIES OR ANY MATTER SUBJECT TO OR RESULTING IN INDEMNIFICATION UNDER THIS AGREEMENT OR OTHERWISE.

FOR AVOIDANCE OF DOUBT, THE SERVICES, INCLUDING ANY ASSOCIATED ASSESSMENT REPORTS OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.

Elevating Your **Web3** Journey

Founded in 2017 by leading academics in the field of Computer Science from both Yale and Columbia University, CertiK is the largest blockchain security company that serves to verify the security and correctness of smart contracts and blockchain-based protocols. Through the utilization of our world-class technical expertise, alongside our proprietary, innovative tech, we're able to support the success of our clients with best-in-class security, all whilst realizing our overarching vision; provable trust for all throughout all facets of blockchain.

